Rubric and evidence for the final assessment

Group 15 • Final assessment preparation • 5 November 2026

The official rubric, page 1, allocates **70% to the final demo and 30% to architecture**. A working demo is a prerequisite. Excellent requires both successful variable-load behavior and an explanation of different architectural bottlenecks under load. A report supports that demonstration; it cannot substitute for it.

What the grading bands require

Component	Fair	Good	Excellent
Demo, 70%	Working interfaces and consistent data satisfying requirements	Also passes variable workload stress with notification latency below 10 seconds	Also keeps up with variable HTTP workload
Architecture, 30%	Interfaces, consistency, and separation of concerns	Distinct responsibilities separated with scalability in mind	Also pinpoints different bottlenecks when load is applied

These are paraphrases, not invented numerical grading rules. The rubric gives no numeric conversion from these bands. It states that individual grades can differ based on contribution and knowledge and that questions may be redirected to any group member.

The ordinary final assessment is assumed here. The rubric's special note limits an additional discretionary resit beyond the midterm/final opportunities to a passing grade for both demo and architecture. If that situation applies, stronger reports cannot remove the policy ceiling.

Requirement to implementation to evidence

Requirement or criterion	Implemented mechanism	Evidence to show and its scope
Observe temperature; do not control it	Pure sensor-group state machine; no actuator interface	`engine.py`; worked exercise in the handbook
Query from configuration onward	Configuration persisted before acknowledgement; metadata endpoint; empty history is valid	API and real-database integration tests
Correct temperature and lifecycle	All allocated sensors for one measurement; inclusive range; explicit phase transitions	Engine tests; original instructor ground truth
Required historical queries	Ordered carry-out arrays, inclusive time bounds, all outside points	202 complete endpoint comparisons in each final native run
Correct transition notifications	Stabilized on entry/re-entry during stabilization; one OutOfRange per running excursion	Notification sets checked against original producer truth
Strict latency	Every required accepted alert must have latency below 10 seconds	Native variable: 703/703, maximum 2.884 s; native fixed: 596/596, maximum 8.958 s
Variable HTTP behavior	Original Rust variable generator plus per-phase validation; independent open-loop profiles	Five native targets passed; independent 25/100/50 passed; independent 200 peak failed
Distinct scaling responsibilities	Ordered scheduler, replaceable executors, authoritative Data, gateway, durable delivery	Architecture diagram, source walkthrough, replica experiment
Pinpoint bottlenecks	Pool wait/occupancy, CPU, queue/partition, outbox, cache, disk metrics	State-write overhead, repeated history work, controller CPU, and shared-host saturation observations
Load-based scaling	Host-side bounded scaling policy; worker discovery and capacity reservations	Policy and scheduler tests; policy is available but not continuously enabled in the demo
Cost overview	One VM plus provisioned storage estimate; Prometheus time series	Configured estimate with gp3 assumption and explicit exclusions
Recoverability	Durable partial groups, checkpoints, outbox, preserved named volumes	Recovery test; recovered database; retained Kafka backlog catch-up

Primary artifacts: final tests, native variable, native fixed, independent passing, independent boundary failure, recovery.

What supports Excellent now

We can demonstrate the required interfaces and consistency, explain separate scaling/failure responsibilities, present a genuine changing-HTTP workload, and identify multiple limiting stages using measurements. Strict boundary tests reject exactly ten seconds. The latest test suite and the live recovery evidence support the final scheduler revision.

The native variable run used 100 simultaneous main experiments, ten sensors each, one-second samples, and an HTTP generator configured to choose targets from 100 through 200. The observed targets were 148, 182, 181, 114, and 106. Each settled phase passed our validation. This does not mean that the random run actually included a 200 phase; the separate fixed run supplies that evidence.

Our local HTTP acceptance rule is at least 95% of the target, with correctness/error checks. **The rubric does not specify 95%.** Native 200 achieved 190.573/s, only 0.573/s above that local threshold. Say this clearly when asked about headroom.

The native producer runs a configured concurrency and sample period with lifecycle changes. These results do not prove every possible variable event-arrival schedule. The independent HTTP test queries live growing histories and measures offered-load omissions/draining, which makes its failed 200 peak relevant even though the native tool passed.

Remaining work with the highest assessment value

Priority	Action for the team	Completion evidence
1	Keep the actual demo healthy and verify external API reachability before assessment	Fresh mode/health/assignment/outbox checks and a laptop/public-route query
2	Rehearse all members across the whole design	Every member explains invariants, one code function, two bottlenecks, and a failed test
3	In a separate test window, repeat representative workloads on the final scheduler revision	Repeated archived runs with exact source/image identity and all outcomes retained
4	Characterize capacity instead of claiming it is unlimited	Longer histories, cold caches, event-rate steps, skew, and soak tests with queue drain
5	Address the measured limiting resource if deployment constraints allow	Controlled comparison on resized/separated infrastructure, including cost

These are follow-up experiments and preparation tasks, not completed tests hidden behind a checklist. No new stress workload was started while preparing these reports.

Claims to avoid and the correct replacement

Overstatement	Defensible statement
All green means the professor passed us	Our added local status panel records our checks on a specified local run; hosted grading needs its own current evidence
The system sustains any 200-rps workload	The native fixed target passed narrowly; the independent live-history 200 peak failed
Microservices make this highly available	Responsibilities are separated, but all services share one VM and disk
Exactly-once notifications	Durable deduplicated database obligations, with external delivery uniqueness requiring receiver idempotency
All outage data was restored	Named-volume data reopened and retained Kafka backlog was processed; broker-expired input cannot be reconstructed from these checks
The scheduler change caused the measured capacity gain	The capacity measurements preceded that change; its separate evidence is tests and backlog recovery
Our cost graph is the AWS bill	It is an estimate using one VM and provisioned disk, with an unconfirmed disk type and stated exclusions

The strongest defense is precise: explain what exists, what a particular experiment measured, what failed, and what you would test next.